

routines.js — Fundamentos

Explicación detallada de la tabla de rutinas y sus conexiones (GymBro App)

Es la **tercera tabla** de la base de datos simulada, y la más compleja estructuralmente: en vez de un array plano de objetos simples (como `users.js` o `exercises.js`), cada rutina tiene **datos anidados en varios niveles** — es un pequeño árbol.

1. La forma de los datos: jerarquía de 4 niveles

```
export const routines = [
  {
    id: 1,
    name: 'Mi Rutina',
    days: [                                // Nivel 1: Días
      {
        id: 1,
        name: 'Día 1',
        blocks: [                          // Nivel 2: Bloques
          {
            id: 1,
            name: 'Bloque 1',
            items: [                        // Nivel 3: Items
              { id: 1, type: 'exercise', exerciseId: 2, series: '4', repetitions: '10',
                weight: '0 kg', comments: '' },
              { id: 2, type: 'rest', time: '1', unit: 'minutos' },
            ],
          },
        ],
      },
    ],
  },
];
```

Cada rutina se descompone así:

- **Rutina** → tiene `id`, `name`, y un array de **días**.
- **Día** → tiene `id`, `name` (ej. “Día 1”), y un array de **bloques**.
- **Bloque** → tiene `id`, `name` (ej. “Bloque 1”), y un array de **items**.
- **Item** → es la unidad más chica, y puede ser de dos tipos distintos según su campo `type`: **type: 'exercise'** (tiene `exerciseId`, la clave foránea a `exercises.js`, más `series`, `repetitions`, `weight`, `comments`) o **type: 'rest'** (tiene `time` y `unit`, ej. “1 minutos” de descanso — no tiene `exerciseId` porque no es un ejercicio, es una pausa dentro de la rutina).

Punto clave para el oral: esto es un patrón de “**tipo discriminado**” (**discriminated union**) — el campo `type` le dice al resto del código cómo interpretar el resto del objeto. Cuando la app recorre `items`, primero mira `item.type` y decide si renderizarlo como ejercicio (con su nombre, series y peso) o como descanso (con su tiempo), en vez de tener dos arrays separados (`exerciseItems` y `restItems`) que perderían el orden en que aparecen dentro del bloque.

2. Los helpers de acceso y mutación (líneas 151-176)

A diferencia de `exercises.js`, que solo tenía una función de lectura, acá hay **cuatro funciones**: una de lectura y tres de escritura, porque las rutinas son datos que el entrenador crea y edita activamente (a diferencia del catálogo fijo de ejercicios).

```
export function getRoutineById(routineId) {
  return routines.find((routine) => routine.id === routineId);
}
```

Mismo patrón .find() que ya se ve en getUserById/getExerciseById.

```
export function createRoutine(days) {
  const routine = {
    id: Math.max(...routines.map((routine) => routine.id)) + 1,
    name: 'Mi Rutina',
    days: JSON.parse(JSON.stringify(days)),
  };
  routines.push(routine);
  return routine;
}
```

- El id nuevo se calcula tomando el **máximo id existente + 1** (simula un autoincremental de base de datos, ya que no hay un motor real que lo genere solo). Mismo patrón que addUser en users.js.
- JSON.parse(JSON.stringify(days)) es una técnica para hacer una **copia profunda** (deep clone) del array days. Es necesaria porque days viene del estado (useState) de la pantalla del entrenador — si se guardara la referencia directa, el objeto en routines y el useState de la pantalla **apuntarían a la misma memoria**, y cualquier cambio posterior en el formulario (antes de guardar) modificaría también los datos ya “persistidos” sin que el entrenador lo confirmara. Clonar rompe esa conexión.

```
export function updateRoutineDays(routineId, days) {
  const routine = getRoutineById(routineId);
  if (!routine) return false;
  routine.days = JSON.parse(JSON.stringify(days));
  return true;
}
```

Actualiza los days de una rutina **ya existente**, con la misma técnica de clonado. Nota que **muta el objeto encontrado directamente** (routine.days = ...) en vez de reemplazar el array routines completo — como routines es un array a nivel de módulo (compartido, no un estado de React), modificar el objeto in-place es suficiente para que el cambio “persista” (dentro de la sesión, en memoria).

3. Cómo se conecta con users.js — el otro lado de la relación

Ya se vio que routines.js → exercises.js es una relación por exerciseId. Acá hay otra relación, en el otro sentido: **users.js → routines.js** mediante el campo id_rutina que tiene cada usuario:

```
// users.js
{ id: 2, name: 'Sol Gaetmank', id_rutina: 1, ... }
```

Y la función que **crea ese vínculo**:

```
// users.js
export function setUserRoutine(userId, routineId) {
  const user = getUserById(userId);
  if (!user) return false;
  user.id_rutina = routineId;
  return true;
}
```

4. Dónde se usa — el flujo completo entrenador → alumno

editar_rutina_trainer.js es la pantalla donde el entrenador arma la rutina de un alumno, y usa **las tres funciones** juntas:

```
import { createRoutine, getRoutineById, updateRoutineDays } from '../data/routines';
```

```
import { setUserRoutine } from '../data/users';

const editedUser = route?.params?.user; // alumno recibido por navegación
const routine = getRoutineById(editedUser?.id_rutina); // busca si ya tiene rutina
const routineId = useRef(routine?.id ?? null); // guarda el id (o null si no tiene)
```

```
const [days, setDays] = useState(routine?.days ?? []); // borrador editable en memoria
```

- Si el alumno **ya tenía** id_rutina, getRoutineById la encuentra y se precarga en el estado days para editar.
- Si **no tenía** (alumno recién registrado, id_rutina: null desde addUser en users.js), routine es undefined, y days arranca como array vacío [].

Al guardar (líneas 468-476):

```
const persistRoutine = () => {
  if (routineId.current === null) {
    const created = createRoutine(days); // 1. crea la rutina nueva
    setUserRoutine(editedUser.id, created.id); // 2. la vincula al alumno (id_rutina)
    routineId.current = created.id; // 3. recuerda el id para futuros guardados
  } else {
    updateRoutineDays(routineId.current, days); // ya existía: solo actualiza
  }
};
```

Este es el flujo de **“crear o actualizar” (upsert)**: la primera vez que un entrenador guarda una rutina para un alumno nuevo, se crea la rutina y se asigna; las veces siguientes, solo se actualiza. routineId es un useRef (no useState) porque su cambio **no necesita** disparar un re-render — solo se usa internamente para decidir qué rama tomar la próxima vez que se guarde, así que un ref es más apropiado que un estado.

mi_rutina_user.js es la pantalla espejo, del lado del alumno, y usa la función de lectura:

```
import { getRoutineById } from '../data/routines';

const routine = getRoutineById(user.id_rutina);
// ...
{routine ? routine.name : 'Todavía no tenés una rutina asignada'}
// ...
{routine?.days.map((day) => ( ... ))}
```

Acá se ve el manejo del caso “todavía no tiene rutina”: si id_rutina es null (alumno recién registrado sin que el entrenador le haya armado nada), getRoutineById(null) no encuentra match y devuelve undefined, y la pantalla muestra el mensaje “Todavía no tenés una rutina asignada” en vez de romper.

5. El grafo completo de relaciones (para dibujar en la defensa)

```
users.js --id_rutina--> routines.js --days->blocks->items.exerciseId--> exercises.js
  ^                               |
  +----- setUserRoutine -----+
      (el entrenador vincula alumno <-> rutina al guardar por primera vez)
```

Resumen para el oral

“routines.js es la tabla más compleja de nuestra base de datos simulada: cada rutina es un árbol de días → bloques → items, donde cada ítem es un ejercicio (con su exerciseId como clave foránea) o un descanso, distinguidos por un campo type. Expone getRoutineById para leer, y createRoutine/updateRoutineDays para el flujo de guardado del entrenador, que hace un patrón de ‘crear o actualizar’ según si el alumno ya tenía una rutina asignada. La relación con el alumno se

guarda en users.js mediante id_rutina, que setUserRoutine actualiza la primera vez que el entrenador guarda. Todas las escrituras clonan los datos con JSON.parse(JSON.stringify(...)) para no compartir referencias de memoria entre el estado de la pantalla y los datos 'persistidos'."